Parcours des écoles d'ingénieurs Polytech (MAM3)

Introduction à la Programmation et à l'Algorithmique

D. Lopez Pacheco, H. Renard et al*



Pour dynamiser et rendre interactif un cours en Cours/TD d'IPA …

Les méthodes et variables statiques

Les membres statique

- une méthode ou variable de classe statique est associée à une classe, pas à l'instance de classe
  - L'accès au membre statique est possible aussi par l'instance, mais **cette pratique est fortement découragée**
- un attribut de classe de type statique est appelé une variable de classe
- Une méthode peut être aussi statique
  - Méthode de classe

```
class A {
    public int val = 0;
    public static void myM() {…}
}

System.out.println(A.val); //erreur
A.myM();

A a = new A();
// possible mais à proscrire
System.out.println(a.val);
a.myM();
```

Les méthodes de classe

- ✓ une méthode de classe (i.e. statique) ne peut pas appeler une méthode d'instance directement
  - ❑ elle doit passer par un objet
  - ❑ une méthode statique peut invoquer des variables de classes ou méthodes de classe directement
- ✓ une méthode de classe ne peut pas utiliser le mot clé « this »
- ✓ une méthode d'instance peut invoquer directement des variables de classe et des méthodes de classe

Héritage

Héritage & Algorithme

Généralités (1/2)

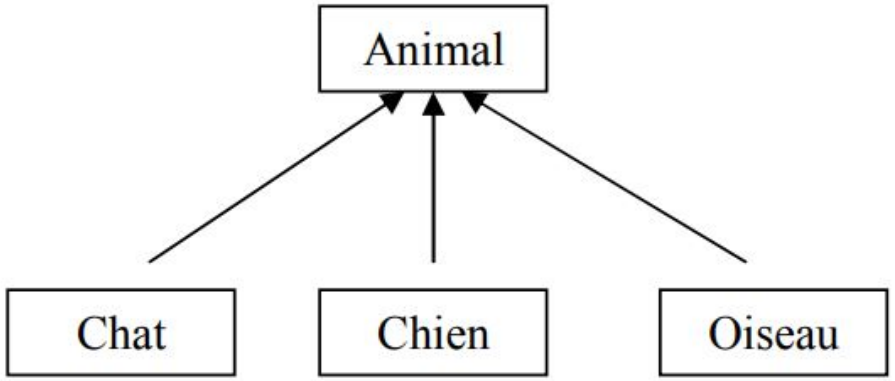
- ✓ L'héritage peut être simple ou multiple;
- ✓ Il représente la relation : ***EST-UN***
  - ❑ Exemple :

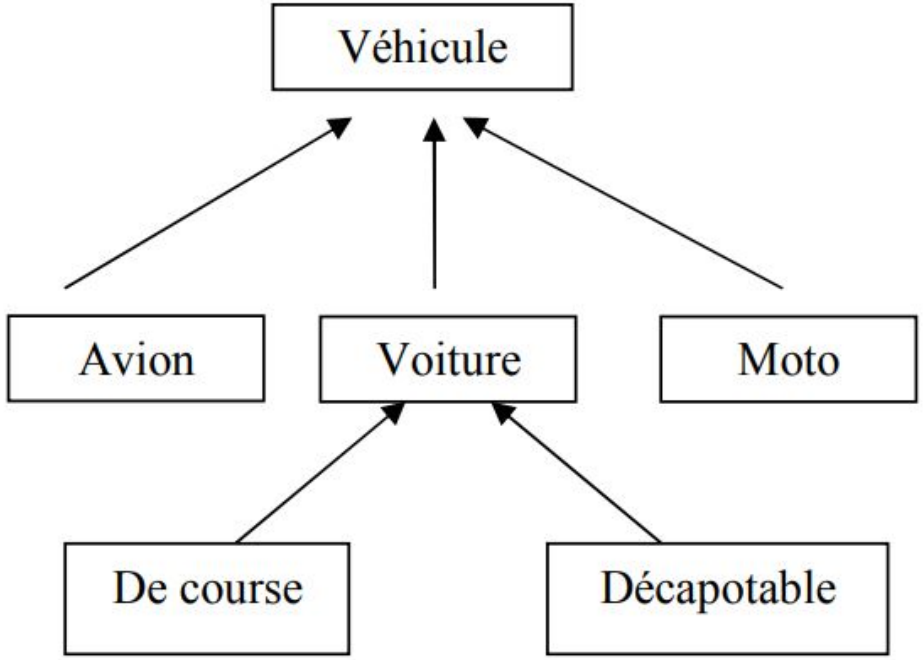
```
Un chat est un animal
Une moto est un véhicule
Un cercle est une forme
```

- ✓ L'héritage est mis en œuvre par la construction de classes dérivées.

Généralités (2/2)

✓ Le graphe de l'héritage est comme suit :





Hiérarchisation



- ✓ La classe dont on dérive est dite *CLASSE DE BASE* :
  - ❑ `Animal` est la classe de base (classe supérieure)
- ✓ Les classes obtenues par dérivation sont dites *CLASSES DÉRIVÉES* :
  - ❑ `Chat`, `Chien` et `Oiseau` sont des classes dérivées (sous-classes)

Intérêt et Utilisation

- ✓ Vision descendante : la possibilité de reprendre intégralement tout ce qui a déjà été fait et de pouvoir l'enrichir;
- ✓ Vision ascendante : la possibilité de regrouper en un seul endroit ce qui est commun à plusieurs.

- ✓ Vers le haut : on regroupe dans une classe ce qui est commun à plusieurs classes;
- ✓ Vers le bas : la classe de base étant définie, on peut la reprendre intégralement pour construire la classe dérivée.

Classe dérivée (1/2)



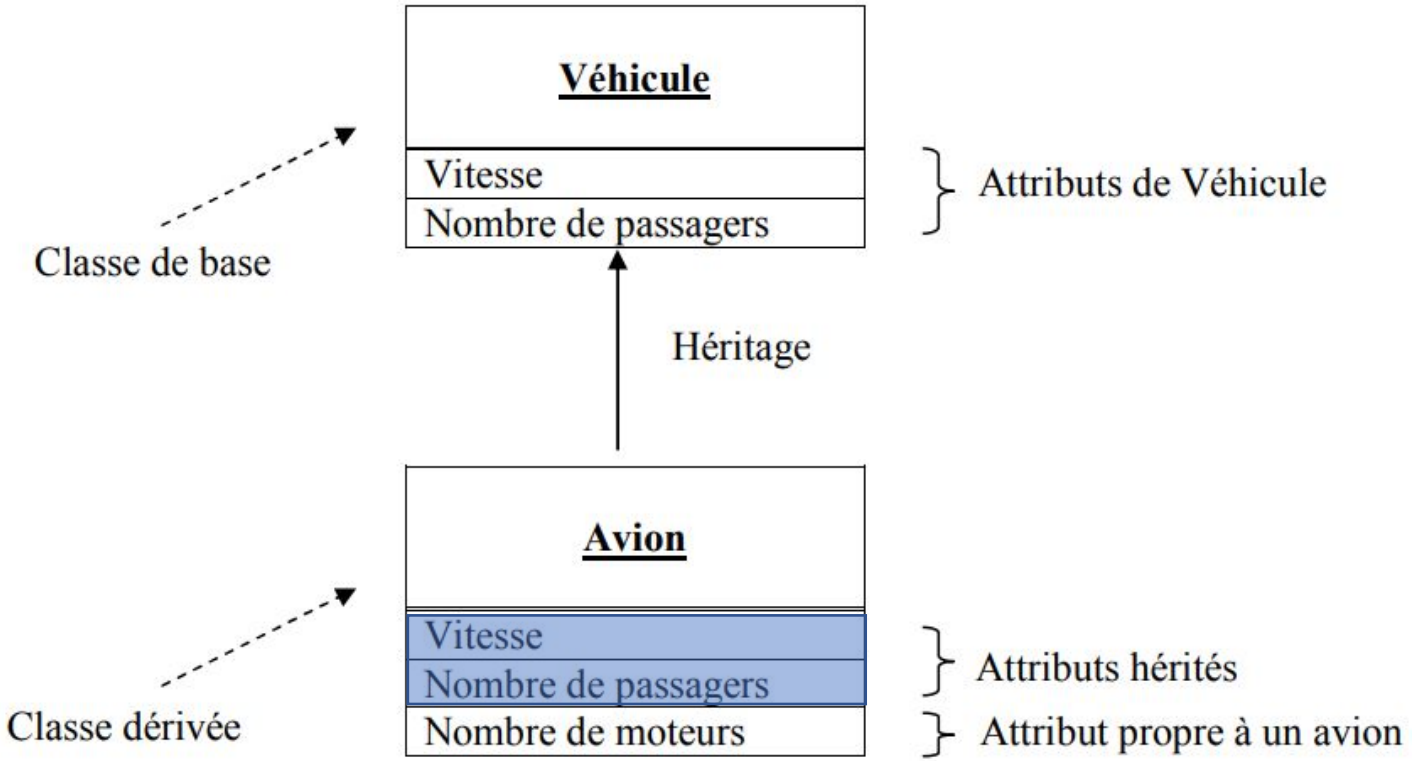
✓ Une classe dérivée modélise un cas particulier de la classe de base, et est enrichie d'informations supplémentaires;

✓ La classe dérivée possède les propriétés suivantes :

- ❑ contient (hérite) les attributs de la classe de base,
- ❑ peut en posséder de nouveaux attributs,
- ❑ possède (hérite) à priori, les méthodes de sa classe de base,
- ❑ peut redéfinir (masquer) certaines méthodes,
- ❑ peut posséder de nouvelles méthodes.

Classe dérivée (2/2)

✓ La classe dérivée hérite des membres de la classe de base :



Syntaxe de l'héritage



- ✓ *control-accès* `class` ClasseDerivee `extends` ClasseDeBase `{/* etc. */}`
  - ❑ *control-accès* : droits d'accès attribués au niveau de la classe
  - ❑ `public` ou bien néant (i.e. rien)
    - public -> tout le monde a accès à la classe
    - Rien -> uniquement le package a accès à la classe

```
class Carre extends Rectangle{
        public Carre(int long){
                super(long, long);
        }
}
```

Constructeur

- ✓ La classe dérivée doit prendre en charge la construction de la classe de base;
- ✓ Exemple :
  - ❑ Pour construire un `Avion`, il faut construire d'abord un `Véhicule`;

- ✓ Le constructeur de la classe de base (`Véhicule`) est donc appelé avant le constructeur de la classe dérivée (`Avion`).
- ✓ Si un constructeur de la classe dérivée appelle explicitement un constructeur de la classe de base, il doit utiliser le mot clé `super`.

Exemple

```
public class A {
      int i;
      /*Constructeur de la classe A*/
      public A(int x){
            i = x;
      }
}
```

```
/* Héritage */
public class B extends A {
      double z;
      /* Constructeur de la classe B */
      public B(int f, double w)
      {
            /* Appel explicite du
constructeur de A, et en 1 ère ligne. */
            super(f);
            z = w;
      }
}
```

Droits d'accès – membres d’une classe

Modificateur	Classe	Package	Sous-classe	Tous
public	O	O	O	O
protected	O	O	O	N
rien	O	O	N	N
private	O	N	N	N

Cacher un membre de classe

- ✓ Lorsqu'un attribut de classe est déclarée à nouveau dans une sous-classe, l'attribut est masqué
- ✓ Une méthode déclarée au niveau de la classe de basse, si elle est statique, la méthode sera cachée si la méthode est redéclarée dans la sous-classe
- ✓Si la méthode cachée est appelé par
  - ❑ la classe de base, c'est la version de la classe de base qui sera utilisée
  - ❑ la sous-classe, c'est la version de la sous-classe qui sera utilisée

Polymorphisme « @Override »

- ✓ Une méthode d'instance (pas statique) peut être déclaré à nouveau dans la sous-classe
  - ❑ Polymorphisme
  - ❑ substitution -> @Override (annotation à insérer avant la redéclaration de la méthode)
- ✓ réutilisation de la méthode de la classe de base par la sous-classe
  - ❑ super.nomMethode()
- ✓ si le constructeur utilisé pour créer un objet est celui de la sous-classe, c'est la version de la sous-classe qui sera utilisée

Caché vs substitution

```
class Animal {
    public static void testClMethod() {
        System.out.println("The static method in Animal");
    }
    public void testInstMethod() {
        System.out.println("The instance method in Animal");
    }
}

public class Cat extends Animal {
    public static void testClMethod() {
        System.out.println("The static method in Cat");
    }
    @Override
    public void testInstMethod() {
        System.out.println("The instance method in Cat");
    }
}
```

```
Animal myAnimal = new Cat();
myAnimal.testClMethod();
myAnimal.testInstMethod();
```

Classes et méthodes finales

- ✓ Si une classe ne doit pas être étendue, une classe doit être déclarée comme « final »
- ✓ une méthode déclarée comme « final » ne peut pas être substituée
- ✓ « final » peut être appliqué aussi sur les attributs de classe
  - ❑ création de constantes

Les classes et méthodes abstraites

Les classes et méthodes abstraits



- ✓ une méthode abstraite est déclaré avec le mot clé « abstract »
  - ❑ une méthode « abstract » est déclarée sans implémentation
    - Sans {}, mais avec un « ; » à la fin
    - public abstract void myMethod(int val);
- ✓Une classe abstraite est déclaré avec le mot clé « abstract »
  - ❑ une classe abstraites ne peut pas être instanciée
  - ❑ une classe abstraite peut contenir ou pas des méthodes abstraites
- ✓ une classe contenant une méthode abstraite doit être déclarée comme abstraite

Questions et remerciements

« *Le pire, quand on fait un discours, ce n'est pas de prendre conscience que vos auditeurs regardent leur montre, c'est le moment où ils se mettent à la secouer pour voir si elle n'est pas arrêtée.* »

Remerciements : Un grand merci à :

Mohamed N. Lokbani